USB Port Session Monitor using Linux Kernel Module

Device Drivers — Mini-Project Report

Team Members
Phanikrishna N M
Chirag Poojary
Tejas Gowda

1. Introduction

The USB LED Monitor is a Linux kernel module developed for the eSrijan DDK v2.1 board running on kernel 6.x. It intercepts every USB device connect and disconnect event on the host system, captures a timestamped log entry for each event (including Vendor ID, Product ID, device name, and event type), persists the log in kernel memory, and provides a physical hardware signal by blinking the board's PGM LED a number of times equal to the current count of connected USB devices.

The project satisfies all five mandatory Linux kernel programming components as specified in the course requirements and operates entirely in kernel space with no userspace daemon or polling loop required.

2. Objectives

- Implement a Linux kernel module that logs every USB device connect and disconnect event on the eSrijan DDK v2.1 board.
- Capture and store timestamped event records — Vendor ID, Product ID, device name, and status — for each USB event.
- Maintain a session log in kernel memory holding up to 50 activity entries.
- Enforce USB event interception entirely in kernel space with no userspace involvement.
- Signal each event on the DDK board's PGM LED: blink N times per cycle where N equals the number of currently connected USB devices.
- Expose the live activity log via a /proc/usb_monitor entry readable from the terminal at any time.
- Demonstrate all five required kernel programming components: proc entry, kernel timer, work queue, atomic operations, and USB subsystem integration.

3. Hardware Platform

The project targets the eSrijan DDK v2.1 development board with the following specifications:

Component	Details
Board	eSrijan DDK v2.1
Microcontroller	LPC3250 (ARM9)
Firmware	LDDK v2.2
USB VID:PID	16C0:05DC (Van Ooijen Technische Informatica)
LED Controlled	PGM (Program Running) LED
Host System	Ubuntu 24.04, Kernel 6.x

The PGM LED on the board is controlled via USB vendor control messages using bRequest=1 (DDK_CMD_SET_LED). wValue=1 turns it ON and wValue=0 turns it OFF. The DDK board is identified by VID:16C0 PID:05DC and excluded from the USB session log — it serves only as the LED controller.

4. System Architecture

The module follows a clean event-driven architecture. The kernel's USB notifier chain delivers connect and disconnect events. LED control is deliberately deferred to a dedicated ordered work queue to avoid executing sleeping operations (usb_control_msg) in atomic or interrupt context. The session array is protected by a spinlock to allow safe concurrent access from both the USB notifier and the /proc reader.

4.1 Component Interaction

Component	Role	Kernel API Used
USB Notifier	Intercepts connect/disconnect events	usb_register_notify
Work Queue	Defers LED blink to process context where USB control transfers are allowed	alloc_ordered_workqueue, queue_work
Session Tracker	Array of 50 usb_session structs tracking each device's name, VID, PID, timestamps	spinlock_t, atomic_t
Blink Engine	Kernel timer FSM producing N blinks per cycle (N = active devices)	timer_setup, mod_timer
Duration Timer	Fires every second to update live duration of active sessions	mod_timer, ktime_get_real_ts64
Proc Entry	Exposes live session log to userspace	proc_create, seq_file
Spinlock	Protects session array from concurrent access	DEFINE_SPINLOCK, spin_lock_irqsave
Mutex	Protects ddk_udev pointer and USB control calls	DEFINE_MUTEX, mutex_lock

4.2 Event Flow

When a USB device is plugged in, the following sequence executes:

- USB subsystem fires USB_DEVICE_ADD via the notifier chain.

- usb_event() runs — reads VID/PID, gets device name via usb_string(), writes session entry under spinlock, increments atomic_t active_count.
- atomic_read(&active_count) is consumed by blink_callback() on its next timer tick to set the blink target N.
- blink_callback() queues LED ON/OFF work items via ddk_led_set(), which allocates led_work structs with GFP_ATOMIC and enqueues them on the ordered workqueue.
- led_work_fn() runs in process context — safely calls usb_control_msg() with the ON or OFF value after acquiring a usb_get_dev() reference.

5. Five Required Kernel Components

5.1 /proc Entry — proc_create / seq_file

A read-only file at /proc/usb_monitor is created using proc_create() with seq_file for safe, paginated output. It displays the session array contents as a formatted activity log — one block per device — showing VID:PID, device name, UTC-formatted insert/remove timestamps, session duration in seconds, and ACTIVE or DISCONNECTED status. The DDK board connection state is shown in the header.

```
$ cat /proc/usb_monitor

=== USB Device Session Monitor + DDK v2.1 PGM LED ===

DDK board   : CONNECTED — LED control active
Active devs : 2  (PGM LED blinks this many times)

Session #1
  Device   : SanDisk Ultra
  VID:PID  : 0781:5591
  Inserted : 2026-04-21 10:14:03 UTC
  Removed  : Still connected
  Duration : 87 sec
  Status   : ACTIVE
```

5.2 Kernel Timer — mod_timer

Two kernel timers are used. A duration_timer fires every second (jiffies + HZ) to update the duration field of all active sessions, so /proc/usb_monitor always shows a live running count without requiring a userspace polling loop. A blink_timer drives the LED blink FSM and re-arms itself after every phase with the appropriate delay (150 ms ON, 150 ms OFF, 800 ms gap).

5.3 Work Queue — alloc_ordered_workqueue / queue_work

An ordered work queue named ddk_led_wq is created with alloc_ordered_workqueue(). Each LED ON or OFF command allocates a led_work item with GFP_ATOMIC (safe from timer context) and queues it. The ordering guarantee ensures that LED ON and OFF commands are always delivered in sequence, preventing LED state corruption from out-of-order execution. The workqueue is fully drained with flush_workqueue() before destroy_workqueue() at module exit.

5.4 Atomic Operations — atomic_t

The active device count is stored in an atomic_t (active_count) rather than a regular integer under a lock. This allows the blink_callback() timer function — which runs in softirq context — to safely read the current count via atomic_read() without acquiring any spinlock, eliminating potential deadlock between the timer and the USB notifier's spinlock-protected code path. atomic_inc() and atomic_dec() are called in the notifier on connect and disconnect respectively.

5.5 USB Subsystem — usb_register_notify / usb_control_msg

The module hooks into the kernel USB notifier chain via usb_register_notify(), which delivers USB_DEVICE_ADD and USB_DEVICE_REMOVE events for every device on every bus system-wide. Device identity strings are read using usb_string() on the manufacturer and product descriptor indices. The DDK board (VID:16C0 PID:05DC) is detected in the notifier and its usb_device pointer stored in ddk_udev under a mutex. PGM LED control is sent via usb_control_msg() with:

```
bmRequestType = 0x40   (vendor | host-to-device | device)
bRequest      = 1      (DDK_CMD_SET_LED)
wValue        = 1 / 0  (LED ON / OFF)
wIndex        = 0, data = NULL, length = 0, timeout = 1000 ms
```

6. PGM LED Blink Pattern

The PGM LED blink pattern is determined by the number of currently active (connected) USB devices, updated atomically on every plug/unplug event:

Active USB Devices	PGM LED Behaviour
Module loaded	LED turned OFF (clean initial state)
0	LED stays OFF permanently
1	Blinks once per cycle — 150 ms ON, 150 ms OFF, 800 ms gap
2	Blinks twice per cycle — ON OFF ON OFF then 800 ms gap
N	Blinks N times per cycle (N = active device count)
Module unloaded	LED turned OFF cleanly

Each blink cycle is driven by a phase-based FSM in blink_callback(). Phase 0 captures the current blink target N from atomic_read(&active_count). Even phases send LED ON (150 ms), odd phases send LED OFF (150 ms). After phase 2N a final OFF command is sent and an 800 ms gap timer fires before the cycle restarts.

7. Project File Structure

File	Purpose
usb_led_monitor.c	Main kernel module — all five components implemented here
Makefile	Builds usb_led_monitor.ko against the running kernel headers

File	Purpose
test_usb_led.sh	Automated test script: loads module, waits 20 s for plug/unplug, dumps session log and dmesg, unloads cleanly
README.md	Build, load, and test instructions; LED fallback GPIO usage

8. Kernel Version Compatibility

Two kernel API differences required compatibility shims during development:

Issue	Old API	Fixed API
Timer teardown	del_timer_sync()	timer_delete_sync() — renamed in kernel 6.7; backcompat shim added via #define
/proc file ops struct	file_operations (< 5.6)	proc_ops struct used for kernel >= 5.6, file_operations for older via LINUX_VERSION_CODE guard

The DMA-safe buffer requirement for usb_control_msg() is satisfied by passing NULL for the data stage in all LED SET calls, which carry no data phase and therefore require no DMA buffer.

9. Build and Run Instructions

9.1 Prerequisites

- Ubuntu 24.04, kernel 6.x
- linux-headers-$(uname -r) installed
- eSrijan DDK v2.1 board connected via USB

9.2 Build

```
cd usb_led_monitor/
make clean && make
```

9.3 Load

```
sudo insmod usb_led_monitor.ko
sudo dmesg | tail -5
```

Expected output:

```
USB LED Monitor: Loading...
USB LED Monitor: DDK board already connected
USB LED Monitor: Ready. cat /proc/usb_monitor
```

9.4 Test

```
# Plug in any USB device -> LED blinks once
# Plug in a second device -> LED blinks twice per cycle
cat /proc/usb_monitor      # view live session log
./test_usb_led.sh          # automated 20-second interactive test
```

```
sudo dmesg | tail -10      # view kernel log
```

9.5 Unload

```
sudo rmmod usb_led_monitor
```

10. Results and Observations

- Module loads successfully on kernel 6.x without warnings or errors.
- DDK board detected at load time via usb_for_each_dev() scan — LED control active immediately.
- All tested USB devices (mouse, pen drive, phone, keyboard) trigger correct CONNECT log entries and PGM LED blinks.
- DISCONNECT events correctly decrement active_count, reducing the blink count on the next timer cycle.
- /proc/usb_monitor correctly shows all buffered sessions with live duration, timestamps, VID:PID, and ACTIVE/DISCONNECTED status.
- Duration values update every second for active sessions without userspace polling.
- Blink count accurately reflects the number of currently connected non-DDK USB devices at all times.
- Module unloads cleanly — PGM LED turns off, workqueue drains fully, both timers cancelled via timer_delete_sync(), /proc entry removed, no kernel warnings.